

Лабораторная работа №2: Основные модели

Мария Улитина

Table of contents

1. Введение	2
2. Модель экспоненциального роста	2
3. Экспоненциальный рост	2
3.1. Инициализация проекта и загрузка пакетов	2
3.2. Определение модели	2
3.3. Первый запуск с параметрами по умолчанию	3
3.4. Визуализация результатов	4
3.5. Анализ результатов	4
3.6. Сохранение всех результатов	5
4. Модель экспоненциального роста с параметрами	5
5. Параметрическое исследование экспоненциального роста	5
5.1. Активация проекта и загрузка пакетов	5
5.2. Определение модели	6
5.3. Определение параметров в Dict	6
5.4. Функция-обертка для запуска одного эксперимента	7
5.5. Запуск базового эксперимента	8
5.6. Визуализация базового эксперимента	8
5.7. Параметрическое сканирование	8
5.8. Запуск всех экспериментов и сбор результатов	9
5.9. Анализ и визуализация результатов сканирования	10
5.10. Бенчмаркинг с разными параметрами	11
5.11. Сохранение всех результатов	13

1. Введение

Исследуются модели Модель экспоненциального роста.

2. Модель экспоненциального роста

3. Экспоненциальный рост

Цель: Исследовать решение уравнения $du/dt = \alpha u$.

3.1. Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab01/project/data/"
```

3.2. Определение модели

Уравнение экспоненциального роста: $\frac{du}{dt} = \alpha u$, $u(0) = u_0$

```
function exponential_growth!(du, u, p, t)
    α = p
    du[1] = α * u[1]
end
```

```
exponential_growth! (generic function with 1 method)
```

3.3. Первый запуск с параметрами по умолчанию

Зададим начальные параметры:

```
u0 = [1.0]          # начальная популяция
α = 0.3             # скорость роста
tspan = (0.0, 10.0) # временной интервал

prob = ODEProblem(exponential_growth!, u0, tspan, α)
sol = solve(prob, Tsit5(), saveat=0.1)
```

```
retcode: Success
Interpolation: 1st order linear
t: 101-element Vector{Float64}:
 0.0
 0.1
 0.2
 0.3
 0.4
 0.5
 0.6
 0.7
 0.8
 0.9
 1.0
 1.1
 1.2
 □
 8.9
 9.0
 9.1
 9.2
 9.3
 9.4
 9.5
 9.6
 9.7
 9.8
 9.9
10.0
u: 101-element Vector{Vector{Float64}}:
 [1.0]
```

```

[1.030454533950446]
[1.0618365551529674]
[1.0941743028794098]
[1.1274968605386673]
[1.1618342327450653]
[1.1972173476990624]
[1.233678057187257]
[1.2712491879500347]
[1.3099646073864084]
[1.3498590188273822]
[1.390968273354968]
[1.4333293964993763]
□
[14.439892233074872]
[14.87962750503581]
[15.33275596650523]
[15.79968870415309]
[16.280848966976613]
[16.776672166300525]
[17.287605875776965]
[17.814109831385352]
[18.35665593143248]
[18.91572823655283]
[19.491822969707854]
[20.08544851618676]

```

3.4. Визуализация результатов

Создаем график и сразу сохраняем без отображения

```
#p = plot(sol, label="u(t)", xlabel="Время t", ylabel="Популяция u",
```

```
title="Экспоненциальный рост ( $\alpha = \$\alpha$ ", lw=2, legend=:topleft)
```

Сохраняем график в PNG формате

```
#savefig(p, plotsdir(script_name, "exponential_growth_α=$α.png"), fmt=:png)
```

3.5. Анализ результатов

Создадим таблицу с данными:

```
df = DataFrame(t=sol.t, u=first.(sol.u))
println("Первые 5 строк результатов:")
println(first(df, 5))
```

Первые 5 строк результатов:

5×2 DataFrame

Row	t	u
	Float64	Float64
1	0.0	1.0
2	0.1	1.03045
3	0.2	1.06184
4	0.3	1.09417
5	0.4	1.1275

Вычислим удвоение популяции:

```
u_final = last(sol.u)[1]
doubling_time = log(2) / α
println("\nАналитическое время удвоения: ", round(doubling_time; digits=2))
```

Аналитическое время удвоения: 2.31

3.6. Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") df
```

4. Модель экспоненциального роста с параметрами

5. Параметрическое исследование экспоненциального роста

5.1. Активация проекта и загрузка пакетов

ИЗМЕНЕНИЕ: Добавлен DrWatson для управления проектом и параметрами

```

using DrWatson
@quickactivate "project" # Активация проекта DrWatson

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools

```

Установка каталогов

```

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

```

```

"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab01/project/data/"

```

5.2. Определение модели

Модель: $du/dt = \alpha \cdot u$

```

function exponential_growth!(du, u, p, t)
    α = p.α # **ИЗМЕНЕНИЕ:** Параметры теперь передаются как именованный кортеж
    du[1] = α * u[1]
end

```

```

exponential_growth! (generic function with 1 method)

```

5.3. Определение параметров в Dict

ОСНОВНОЕ ИЗМЕНЕНИЕ: Все параметры собраны в Dict для систематизации

Базовый набор параметров (один эксперимент)

```

base_params = Dict(
    :u0 => [1.0],          # начальная популяция
    :α => 0.3,             # скорость роста
    :tspan => (0.0, 10.0), # интервал времени
    :solver => Tsit5(),     # метод решения
    :saveat => 0.1,        # шаг сохранения результатов
    :experiment_name => "base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println("  $key = $value")
end

```

Базовые параметры эксперимента:

```

α = 0.3
u0 = [1.0]
saveat = 0.1
solver = Tsit5{typeof(OrdinaryDiffEqCore.trivial_limiter!), typeof(OrdinaryDiffEqCore.ODEProblem{...})}
experiment_name = base_experiment
tspan = (0.0, 10.0)

```

5.4. Функция-обертка для запуска одного эксперимента

ИСПРАВЛЕНИЕ: Возвращаем Dict со строковыми ключами

```

function run_single_experiment(params::Dict)
    @unpack u0, α, tspan, solver, saveat = params
    prob = ODEProblem(exponential_growth!, u0, tspan, (α=α,))
    sol = solve(prob, solver; saveat=saveat)
    final_population = last(sol.u)[1] # Анализ результатов
    doubling_time = log(2) / α
    return Dict(
        "solution" => sol,
        "time_points" => sol.t,
        "population_values" => first.(sol.u),
        "final_population" => final_population,
        "doubling_time" => doubling_time,
        "parameters" => params # Сохраняем исходные параметры
    ) # Используем строки как ключи для совместимости с DrWatson
end

```

```
run_single_experiment (generic function with 1 method)
```

5.5. Запуск базового эксперимента

ИЗМЕНЕНИЕ: Используем `produce_or_load` для автоматического кэширования

```
data, path = produce_or_load(
    datadir(script_name, "single"),      # Папка для сохранения
    base_params,                          # Параметры эксперимента
    run_single_experiment,                # Функция для выполнения
    prefix = "exp_growth",               # Префикс имени файла
    tag = false,                          # Не добавлять git-тег
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println("  Финальная популяция: ", data["final_population"])
println("  Время удвоения: ", round(data["doubling_time"]; digits=2))
println("  Файл результатов: ", path)
```

Результаты базового эксперимента:

Финальная популяция: 20.08544851618676

Время удвоения: 2.31

Файл результатов: /home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-modeling/labs/lab01/project/data/single/exp_growth_experiment_

5.6. Визуализация базового эксперимента

5.7. Параметрическое сканирование

НОВАЯ СЕКЦИЯ: Исследование влияния параметра α

Сетка параметров для сканирования

```
param_grid = Dict(
    :u0 => [[1.0]],      # фиксируем начальное условие
    :α => [0.1, 0.3, 0.5, 0.8, 1.0], # исследуемые значения скорости роста
    :tspan => [(0.0, 10.0)], # фиксируем интервал времени
    :solver => [Tsit5()],   # фиксируем метод решения
    :saveat => [0.1],       # фиксируем шаг сохранения
)
```



```

        :experiment_name => ["parametric_scan"]
    )

```

```

Dict{Symbol, Vector} with 6 entries:
  :α           => [0.1, 0.3, 0.5, 0.8, 1.0]
  :u0          => [[1.0]]
  :saveat      => [0.1]
  :solver      => [Tsit5{typeof(trivial_limiter!), typeof(trivial_limiter!)...}
  :experiment_name => ["parametric_scan"]
  :tspan       => [(0.0, 10.0)]

```

Генерация всех комбинаций параметров

```

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые значения α: ", param_grid[:α])
println("="^60)

```

```

=====
ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ
Всего комбинаций параметров: 5
Исследуемые значения α: [0.1, 0.3, 0.5, 0.8, 1.0]
=====

```

5.8. Запуск всех экспериментов и сбор результатов

НОВАЯ СЕКЦИЯ: Автоматический запуск и сохранение всех вариантов

```

all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Пропеccc: $i/$(length(all_params)) | α = $(params[:α])")

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"), # Данные

```

```

        params,                                # Текущий набор параметров
        run_single_experiment,                 # Функция для выполнения
        prefix = "scan",                       # Префикс имени файла
        tag = false,                           #
        verbose = false                        # Не выводить подробности для каждого запуска
    ) # Автоматическое сохранение/загрузка каждого эксперимента

    result_summary = merge(
        params,
        Dict(
            :final_population => data["final_population"],
            :doubling_time => data["doubling_time"],
            :filepath => path # Путь к сохраненным данным
        )
    ) # Сохраняем сводные результаты (используем символы для параметров, но данные)

    push!(all_results, result_summary)

    df = DataFrame(
        t = data["time_points"],
        u = data["population_values"],
        α = fill(params[:α], length(data["time_points"]))
    ) # Сохраняем полные данные для визуализации
    push!(all_dfs, df)
end

```

```

Прогресс: 1/5 | α = 0.1
Прогресс: 2/5 | α = 0.3
Прогресс: 3/5 | α = 0.5
Прогресс: 4/5 | α = 0.8
Прогресс: 5/5 | α = 1.0

```

5.9. Анализ и визуализация результатов сканирования

НОВАЯ СЕКЦИЯ: Сравнительный анализ всех экспериментов

Сводная таблица результатов

```

results_df = DataFrame(all_results)
println("\nСводная таблица результатов:")
println(results_df[:, [:α, :final_population, :doubling_time]])

```

Сводная таблица результатов:

5×3 DataFrame

Row	α Float64	final_population Float64	doubling_time Float64
1	0.1	2.71828	6.93147
2	0.3	20.0854	2.31049
3	0.5	148.409	1.38629
4	0.8	2980.57	0.866434
5	1.0	22021.0	0.693147

Сравнительный график всех траекторий

5.10. Бенчмаркинг с разными параметрами

ИЗМЕНЕНИЕ: Бенчмаркинг для разных значений α

```
println("\n" * "="^60)
println("Бенчмаркинг для разных значений  $\alpha$ ")
println("="^60)

benchmark_results = []
for  $\alpha$ _value in param_grid[: $\alpha$ ]

    bench_params = Dict(
        :u0 => [1.0],
        : $\alpha$  =>  $\alpha$ _value,
        :tspan => (0.0, 10.0),
        :solver => Tsit5(),
        :saveat => 0.1
    ) # Подготавливаем параметры для бенчмарка

    function benchmark_run() # Функция для бенчмарка
        prob = ODEProblem(exponential_growth!,
            bench_params[:u0],
            bench_params[:tspan],
            ( $\alpha$ =bench_params[: $\alpha$ ],))
        return solve(prob, bench_params[:solver];
            saveat=bench_params[:saveat])
    end
```

```
println("\nБенчмарк для  $\alpha$  =  $\alpha\_value$ :")
b = @benchmark $benchmark_run() samples=100 evals=1 # Запуск бенчмарка
push!(benchmark_results, ( $\alpha$ = $\alpha\_value$ , time=median(b).time/1e9)) # время в сек

println(" Среднее время: ", round(median(b).time/1e9; digits=4), " сек")
end
```

```
=====
Бенчмаркинг для разных значений  $\alpha$ 
=====
```

Бенчмарк для α = 0.1:
Среднее время: 0.0001 сек

Бенчмарк для α = 0.3:
Среднее время: 0.0001 сек

Бенчмарк для α = 0.5:
Среднее время: 0.0001 сек

Бенчмарк для α = 0.8:
Среднее время: 0.0001 сек

Бенчмарк для α = 1.0:
Среднее время: 0.0001 сек

График зависимости времени вычисления от α

```
bench_df = DataFrame(benchmark_results)
```

	α	time
	Float64	Float64
1	0.1	8.48045e-5
2	0.3	6.4241e-5
3	0.5	8.6232e-5
4	0.8	5.9938e-5
5	1.0	7.4145e-5

5.11. Сохранение всех результатов

НОВАЯ СЕКЦИЯ: Сохранение сводных данных для последующего анализа

```
println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
println("\nРезультаты сохранены в:")
println("  • data/$(script_name)/single/           - базовый эксперимент")
println("  • data/$(script_name)/parametric_scan/    - параметрическое сканирова")
println("  • data/$(script_name)/all_results.jld2      - сводные данные")
println("  • plots/$(script_name)/                   - все графики")
println("  • data/$(script_name)/all_plots.jld2       - объекты графиков")
println("\nДля анализа результатов используйте:")
println("  using JLD2, DataFrames")
println("  @load \"data/$(script_name)/all_results.jld2\"")
println("  println(results_df)")
```

```
=====
ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА
=====
```

Результаты сохранены в:

- data//single/ - базовый эксперимент
- data//parametric_scan/ - параметрическое сканирование
- data//all_results.jld2 - сводные данные
- plots// - все графики
- data//all_plots.jld2 - объекты графиков

Для анализа результатов используйте:

```
using JLD2, DataFrames
@load "data//all_results.jld2"
println(results_df)
```